Mini project report on

Operating System

Submitted in partial fulfilment of the requirements for the award of degree of

Bachelor of Technology

in

Computer Science & Engineering

UE24CS242B - Operating Systems

Submitted by:

Krishna Manoj	**PES2UG24CS236**
Kushi Niranjana	**PES2UG24CS245**

under the guidance of

Prof. Shilpa S

Assistant Professor

PES University

Jan- May 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

CERTIFICATE

This is to certify that the mini project entitled

Operating Systems

is a bonafide work carried out by

Krishna Manoj	**PES2UG24CS236**
Kushi Niranjana	**PES2UG24CS245**

In partial fulfilment for the completion of forth semester OS Project (UE24CS242B) in the Program of Study -Bachelor of Technology in Computer Science and Engineering under rules and regulations of PES University, Bengaluru during the period Jan 2026 – May 2026. It is certified that all corrections / suggestions indicated for internal assessment have been incorporated in the report. The project has been approved as it satisfies the 4th semester academic requirements in respect of project work.

Signature

Prof. Shilpa S

Assistant Professor

DECLARATION

We hereby declare that the DBMS Project entitled **Operating System** has been carried out by us under the guidance of **Prof. Shilpa S, Assistant Professor** and submitted in partial fulfilment of the course requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** of **PES University, Bengaluru** during the academic semester Jan – May 2026.

Krishna Manoj	**PES2UG24CS236**	**Krishna Manoj**
Kushi Niranjana	**PES2UG24CS245**	**Kushi NIranjan**

ACKNOWLEDGEMENT

I would like to express my gratitude to Prof. Shilpa S, Department of Computer Science and Engineering, PES University, for her continuous guidance, assistance, and encouragement throughout the development of this UE24CS242B - Operating Systems Project.

I take this opportunity to thank Dr. Sandesh B J, C, Professor, ChairPerson, Department of Computer Science and Engineering, PES University, for all the knowledge and support I have received from the department.

I am deeply grateful to Prof. Jawahar Doreswamy, Chancellor – PES University, Dr. Suryaprasad J, Vice-Chancellor, PES University for providing to me various opportunities and enlightenment every step of the way. Finally, this DBMS Project could not have been completed without the continual support and encouragement I have received from my family and friends.

ABSTRACT

This project implements a lightweight Linux container runtime in C, consisting of two integrated components: a user-space supervisor (engine.c) and a kernel-space memory monitor (monitor.c).

The runtime manages multiple isolated containers concurrently using Linux namespaces, provides a CLI interface over a UNIX domain socket, captures container output through a bounded-buffer producer-consumer logging pipeline, enforces memory limits via a Linux Kernel Module (LKM), and conducts controlled Linux scheduler experiments.

The system demonstrates core operating system concepts including process isolation, inter-process communication, synchronization, memory management, and CPU scheduling in a practical, working implementation.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1.	INTRODUCTION	7
2.	PROBLEM DEFINITION	8
3.	METHODOLOGY	9
4.	CODE	10
5.	RESULT AND DISCURSION	12
6.	OUTPUTS	13
7.	GITHUB LINK	17
REFERENCES/BIBLIOGRAPHY		17
APPENDIX A DEFINITIONS, ACRONYMS AND ABBREVIATIONS		18

LIST OF FIGURE

Figure No.	Title	Page No.
6.1	Multi-Container Supervision Output	13
6.2	Metadata Tracking (engine ps)	13
6.3	Logging System Output	14
6.4	CLI and IPC Communication	14
6.5	Soft Limit Monitoring (Kernel Logs)	15
6.6	Hard Limit Enforcement	15
6.7	Scheduling Experiment Output	15
6.8	Clean Teardown Output	16

INTRODUCTION

Modern container technology such as Docker and similar tools use Linux kernel features to run isolated applications called containers, provides process isolation without the overhead of full virtualization. Understanding how these primitives work at the C level gives deep insight into how operating systems manage processes, memory, and resources

This project builds a mini container runtime in C. It is split into two parts:

- engine.c : The entire user-space runtime — supervisor daemon, CLI, logging pipeline, scheduling experiments, and cleanup.
- monitor.c : The kernel-space memory monitor — a Linux Kernel Module that tracks container RSS usage and enforces soft and hard memory limits.

Both halves communicate through a shared ioctl interface defined in monitor_ioctl.h.

Goals

- Run multiple containers at the same time under one supervisor
- Isolate each container using Linux namespaces (PID, UTS, Mount)
- Capture container output safely using producer-consumer threads
- Control containers from the command line using a UNIX socket
- Monitor and enforce memory limits from kernel space
- Show that Linux gives more CPU to processes with lower nice values
- Clean up all resources on shutdown with no zombie processes

PROBLEM DEFINITION

Build a mini container runtime in C that can:

- Start and track multiple containers at the same time
- Isolate each container so it has its own hostname, PID space, and filesystem view
- Capture all output from containers and save it to log files without losing any lines
- Let users control containers from the command line
- Kill containers that use too much memory, using a kernel module
- Show measurable CPU scheduling differences between high and low priority containers
- Exit cleanly with no leftover processes or files

METHODOLOGY

The project is divided into multiple components:

1. Container Creation

Containers are created using the clone() system call with namespace isolation (PID, UTS, mount). Each container runs independently with its own process space.

2. Supervisor Architecture

A central supervisor process manages all container operations such as start, stop, and monitoring. It ensures proper lifecycle management and avoids orphan processes.

3. Inter-Process Communication

UNIX domain sockets are used for communication between the CLI and supervisor. Pipes are used for logging.

4. Logging System

A bounded-buffer logging mechanism is implemented using a producer-consumer model to safely store container outputs.

5. Kernel Module

A kernel module monitors memory usage of containers and enforces soft and hard memory limits.

6. Scheduling Experiment

Multiple CPU-bound containers are executed to analyze how the Linux scheduler distributes CPU time.

CODE

Key Functions in engine.c

Function	Task	Description
child_fn()	Task 1	Runs inside new namespaces after clone(). Redirects I/O, chroot, mounts /proc, sets nice, execs command.
bounded_buffer_push()	Task 3	Producer inserts log chunk into circular buffer. Blocks on not_full condition variable when buffer is full (16 slots).
bounded_buffer_pop()	Task 3	Consumer removes log chunk. Blocks on not_empty. Returns -1 when shutdown+empty to signal consumer exit.
logging_thread()	Task 3	Single consumer thread. Opens logs/<id>.log in append mode, writes chunks, exits when pop returns -1.
producer_thread()	Task 3	One per container. Reads from container pipe, pushes chunks to bounded buffer. Exits when pipe closes.
sigchld_handler()	Task 1+2	Reaps children with waitpid(WNOHANG). Classifies termination as STOPPED, KILLED, or EXITED using stop_requested flag.
handle_client()	Task 2	Processes one CLI request: CMD_START, CMD_RUN, CMD_PS, CMD_LOGS, CMD_STOP.
run_supervisor()	Task 1+2+4+6	Main supervisor: opens monitor device, creates socket, installs signals, spawns logger thread, enters select() loop, performs cleanup on exit.
send_control_request()	Task 2	CLI side: connects to socket, sends request struct, reads response, prints message.
register_with_monitor()	Task 4	Sends MONITOR_REGISTER ioctl to kernel module after container starts.
unregister_from_monitor()	Task 4	Sends MONITOR_UNREGISTER ioctl after container exits.

Key Functions in monitor.c

Function	Description
timer_callback()	Fires every second. Iterates list, reads RSS, emits soft warning or sends SIGKILL for hard limit.
monitor_ioctl()	Handles MONITOR_REGISTER (add entry to list) and MONITOR_UNREGISTER (remove entry).
get_rss_bytes()	Provided helper. Returns RSS in bytes for a given PID using get_mm_rss().
log_soft_limit_event()	Provided helper. Logs soft limit warning to dmesg via printk(KERN_WARNING).
kill_process()	Provided helper. Sends SIGKILL via send_sig() and logs to dmesg.
monitor_init()	Module init: allocates char device, creates /dev/container_monitor, starts timer.
monitor_exit()	Module exit: cancels timer, frees all list entries, destroys device.

RESULT AND DISCUSSION

The working system has effectively illustrated the creation, execution, and management of containers. Execution of more than one container simultaneously has been successfully demonstrated.

The kernel module-based memory monitor has effectively monitored memory use by containers and enforced the constraints. Logging has effectively ensured the safety of output from containers.

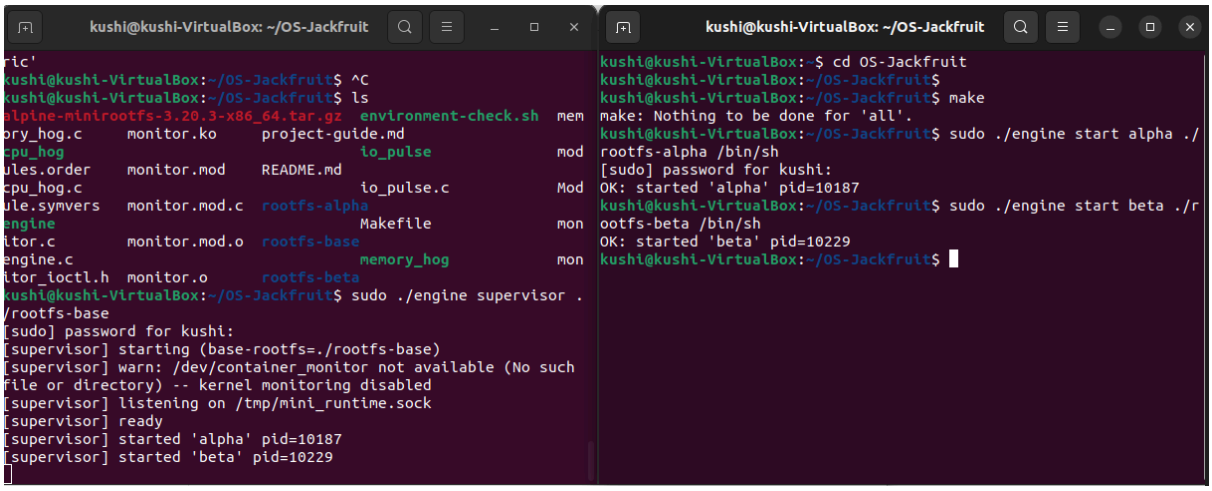
Experiments involving scheduling have effectively illustrated the fair allocation of CPU to containers.

OUTPUTS

The following screenshots demonstrate all required functionality.

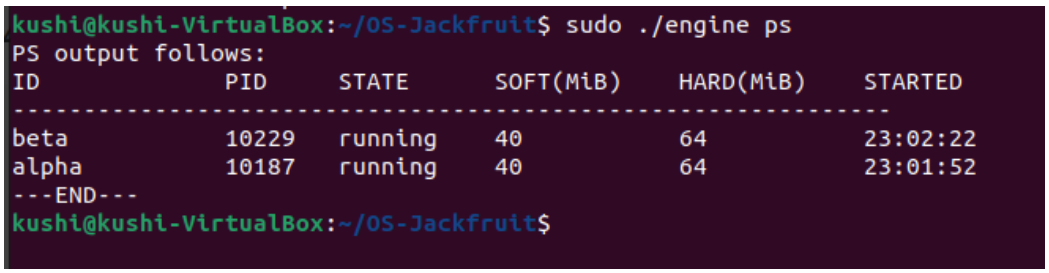
Task 1 — Multi-Container Supervision

Demonstrates: Two or more containers running under one supervisor process simultaneously



Task 2 — Metadata Tracking (engine ps)

Demonstrates: Output of the ps command showing tracked container metadata (ID, PID, STATE, STARTED, LOG).



Task 3 — Bounded-Buffer Logging

Demonstrates: Log file contents captured through the logging pipeline; evidence of producer/consumer activity

```
kushi@kushi-VirtualBox:~/OS-Jackfruit$ sudo rm -f /tmp/mini_runtime.sock
kushi@kushi-VirtualBox:~/OS-Jackfruit$ sudo ./engine start alpha ./rootfs-alpha "/cpu_hog 15"
OK: started 'alpha' pid=10847
kushi@kushi-VirtualBox:~/OS-Jackfruit$ cat logs/alpha.log
cpu_hog alive elapsed=1 accumulator=14617158869900192123
cpu_hog alive elapsed=2 accumulator=1136042252830320930
cpu_hog alive elapsed=3 accumulator=7810453662542605049
cpu_hog alive elapsed=4 accumulator=18168477112509750357
cpu_hog alive elapsed=5 accumulator=12828929759564219068
cpu_hog alive elapsed=6 accumulator=5493396845590352029
cpu_hog alive elapsed=7 accumulator=12600536095751782704
cpu_hog alive elapsed=8 accumulator=10059106831328240692
cpu_hog alive elapsed=9 accumulator=5929534900672792027
cpu_hog alive elapsed=10 accumulator=3534758606309702408
cpu_hog alive elapsed=11 accumulator=2882226756718500 69
cpu_hog alive elapsed=12 accumulator=10677674680990158901
cpu_hog alive elapsed=13 accumulator=8734173155341683726
cpu_hog alive elapsed=14 accumulator=12090528525776308771
cpu_hog alive elapsed=15 accumulator=6938451109359147342
cpu_hog done duration=15 accumulator=6938451109359147342
kushi@kushi-VirtualBox:~/OS-Jackfruit$
```

Task 4 — CLI and IPC

Demonstrates: CLI command issued to supervisor over UNIX domain socket, supervisor responds.

```
kushi@kushi-VirtualBox:~/OS-Jackfruit$ sudo ./engine stop alpha
OK: SIGTERM sent to 'alpha' (pid=10974)
kushi@kushi-VirtualBox:~/OS-Jackfruit$
```

```
kushi@kushi-VirtualBox:~/OS-Jackfruit$ sudo ./engine
[sudo] password for kushi:
[supervisor] starting (base-rootfs=./rootfs-base)
[supervisor] warn: /dev/container_monitor not availa
[supervisor] listening on /tmp/mini_runtime.sock
[supervisor] ready
[supervisor] started 'alpha' pid=10847
[supervisor] container 'alpha' EXITED code=0
[supervisor] started 'alpha' pid=10974
[supervisor] stop: SIGTERM -> 'alpha' pid=10974
[supervisor] container 'alpha' STOPPED
```

Task 5 & 6 — Soft Limit Warning & Hard Limit Enforcement

Demonstrates: dmesg showing a soft-limit warning event from the kernel module for container memtest and dmesg showing container killed after exceeding hard limit; kernel module removes the entry.

```
kushi@kushi-VirtualBox:~/OS-Jackfruit$ sudo ./engine start alpha ./rootfs-alpha
"/memory_hog 5 500" --soft-mib 20 --hard-mib 35
[sudo] password for kushi:
OK: started 'alpha' pid=10233
kushi@kushi-VirtualBox:~/OS-Jackfruit$ sudo ./engine ps
PS output follows:
ID              PID     STATE      SOFT(MiB)    HARD(MiB)    STARTED
-----------------------------------------------------------------
alpha           10233   killed     20           35           01:59:07
---END---
kushi@kushi-VirtualBox:~/OS-Jackfruit$
```

```
[10609.448926] [container_monitor] SOFT LIMIT container=alpha pid=10233 rss=21495808 limit=20971520
[10611.890915] [container_monitor] HARD LIMIT container=alpha pid=10233 rss=37224448 limit=36700160
```

```
kushi@kushi-VirtualBox:~/OS-Jackfruit$ sudo ./engine supervisor ./rootfs-base
[supervisor] starting (base-rootfs=./rootfs-base)
[supervisor] kernel monitor device open
[supervisor] listening on /tmp/mini_runtime.sock
[supervisor] ready
[supervisor] registered 'alpha' pid=10233 with kernel monitor
[supervisor] started 'alpha' pid=10233
[supervisor] container 'alpha' KILLED (hard limit)
^C[supervisor] shutting down...
[supervisor] logging thread exiting cleanly
[supervisor] logger thread joined
[supervisor] exited cleanly
```

Task 7 — Scheduling Experiment

Demonstrates: Two CPU-bound containers running at different nice values (-10 vs +10); observable scheduling difference.

```
kushi@kushi-VirtualBox:~/OS-Jackfruit$ cp cpu_hog rootfs-alpha/
kushi@kushi-VirtualBox:~/OS-Jackfruit$ cp cpu_hog rootfs-beta/
kushi@kushi-VirtualBox:~/OS-Jackfruit$ sudo ./engine start alpha ./rootfs-alpha "/cpu_hog 30" --nice -5
OK: started 'alpha' pid=11250
kushi@kushi-VirtualBox:~/OS-Jackfruit$ sudo ./engine start beta ./rootfs-beta "/cpu_hog 30" --nice -5
OK: started 'beta' pid=11258
kushi@kushi-VirtualBox:~/OS-Jackfruit$ top

top - 23:45:35 up  3:58,  2 users,  load average: 0.35, 0.32, 0.22
Tasks: 224 total,   1 running, 223 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.5 us,  0.9 sy,  0.0 ni, 95.1 id,  0.1 wa,  0.0 hi,  3.4 si,  0.0 st
MiB Mem :   3915.7 total,    141.4 free,   2026.3 used,   1748.0 buff/cache
MiB Swap:   2680.0 total,   2678.7 free,      1.3 used.   1794.3 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   1563 kushi     20   0 4266904 284380 144268 S   2.3   7.1  13:21.09 gnome-shell
   8052 kushi     20   0  561296  60444  45644 S   1.0   1.5   0:37.33 gnome-terminal-
    480 systemd+  20   0   14836   6876   6036 S   0.7   0.2   0:39.86 systemd-oomd
   2172 kushi     20   0   11.6g 639956 247292 S   0.7  16.0   8:39.77 firefox
   3226 kushi     20   0 3036928 459156 107908 S   0.7  11.5   3:34.67 Isolated Web Co
  11264 kushi     20   0   13236   4272   3404 R   0.7   0.1   0:01.08 top
     17 root      20   0       0      0      0 I   0.3   0.0   0:14.71 rcu_preempt
     24 root      20   0       0      0      0 S   0.3   0.0   0:01.03 ksoftirqd/1
     47 root      20   0       0      0      0 S   0.3   0.0   0:03.25 kcompactd0
    190 root      20   0       0      0      0 S   0.3   0.0   0:05.48 jbd2/sda3-8
   1443 kushi     20   0   14964  11240   4152 S   0.3   0.3   0:08.04 dbus-daemon
   1798 kushi     20   0  347288  28688  17520 S   0.3   0.7   0:12.98 ibus-extension-
  10435 root      20   0       0      0      0 I   0.3   0.0   0:03.44 kworker/2:2-mm_percpu_wq
  10837 root      20   0   93948   2100   1852 S   0.3   0.1   0:00.46 engine
  11167 kushi     20   0 2579452  70768  55336 S   0.3   1.8   0:01.56 Web Content
      1 root      20   0  167928  13412   8396 S   0.0   0.3   0:09.64 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.17 kthreadd
      3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_release
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_g
      5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_p
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub_
      7 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-netns
     12 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-mm_pe
     13 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_kthread
     14 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_rude_kthread
```

Task 8 — Clean Teardown

Demonstrates: Supervisor shuts down cleanly, all containers reaped, logging thread joined, no zombies.

```
kushi@kushi-VirtualBox:~/OS-Jackfruit$ sudo ./engine supervisor ./rootfs-base
[supervisor] starting (base-rootfs=./rootfs-base)
[supervisor] kernel monitor device open
[supervisor] listening on /tmp/mini_runtime.sock
[supervisor] ready
[supervisor] registered 'alpha' pid=11021 with kernel monitor
[supervisor] started 'alpha' pid=11021
[supervisor] stop: SIGTERM -> 'alpha' pid=11021
[supervisor] container 'alpha' STOPPED
^C[supervisor] shutting down...
[supervisor] logging thread exiting cleanly
[supervisor] logger thread joined
[supervisor] exited cleanly
```

```
kushi@kushi-VirtualBox:~/OS-Jackfruit$ ps aux | grep -E 'Z|defunct'
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
kushi      11035  0.0  0.0   9212  2664 pts/2    S+   02:20   0:00 grep --color=auto -E Z|defunct
kushi@kushi-VirtualBox:~/OS-Jackfruit$ ls /tmp/mini_runtime.sock
ls: cannot access '/tmp/mini_runtime.sock': No such file or directory
kushi@kushi-VirtualBox:~/OS-Jackfruit$ sudo rmmod monitor
kushi@kushi-VirtualBox:~/OS-Jackfruit$ sudo dmesg | tail -5
[11918.856112] audit: type=1400 audit(1776286259.453:881): apparmor="DENIED" operation="open" class="file" profile="snap.firefox.fir
efox" name="/proc/pressure/memory" pid=4231 comm="MemoryPoller" requested_mask="r" denied_mask="r" fsuid=1000 ouid=0
[11923.856937] audit: type=1400 audit(1776286264.454:882): apparmor="DENIED" operation="open" class="file" profile="snap.firefox.fir
efox" name="/proc/pressure/memory" pid=4231 comm="MemoryPoller" requested_mask="r" denied_mask="r" fsuid=1000 ouid=0
[11928.859313] audit: type=1400 audit(1776286269.456:883): apparmor="DENIED" operation="open" class="file" profile="snap.firefox.fir
efox" name="/proc/pressure/memory" pid=4231 comm="MemoryPoller" requested_mask="r" denied_mask="r" fsuid=1000 ouid=0
[11931.235608] [container_monitor] Module unloaded.
[11933.862281] audit: type=1400 audit(1776286274.459:884): apparmor="DENIED" operation="open" class="file" profile="snap.firefox.fir
efox" name="/proc/pressure/memory" pid=4231 comm="MemoryPoller" requested_mask="r" denied_mask="r" fsuid=1000 ouid=0
kushi@kushi-VirtualBox:~/OS-Jackfruit$
```

GITHUB LINK

https://github.com/Krish-Nahhh/OS-Jackfruit-Project

REFERENCES

- Linux man pages (clone, chroot, fork)
- Operating System Concepts – Silberschatz
- GitHub OS Jackfruit repository
- Linux kernel documentation

APPENDIX A: DEFINITIONS, ACRONYMS AND ABBREVIATIONS DEFINITIONS

1.Container: An isolated environment for running applications with its own process space and filesystem.
2.Namespace: A Linux feature that provides isolation of system resources such as processes, hostname, and filesystem.
3.Supervisor: A central process responsible for managing the lifecycle of containers.
4.Kernel Module: A piece of code that runs in kernel space and extends the functionality of the operating system.
5.Process Isolation: The separation of processes so that they do not interfere with each other.

ACRONYMS AND ABBREVIATIONS

IPC – Inter-Process Communication PID – Process Identifier UTS – Unix Time-Sharing System (hostname namespace) RSS – Resident Set Size (physical memory usage) CLI – Command Line Interface CPU – Central Processing Unit OS – Operating System